STM32 Bootloader

Concepts & Implementation Guide

A generic, hardware-independent reference for embedded engineers

Applies to all STM32 families: F0, F1, F4, F7, H7, L4, G0, G4, U5 …

1. What is a Bootloader?

A **bootloader** is a small piece of firmware that runs **before your main application** every time the MCU powers on. Its primary job is to check whether a firmware update is available and, if so, to write the new firmware into the MCU's internal flash memory. Once the update is complete (or if no update is available), it hands control over to the application.

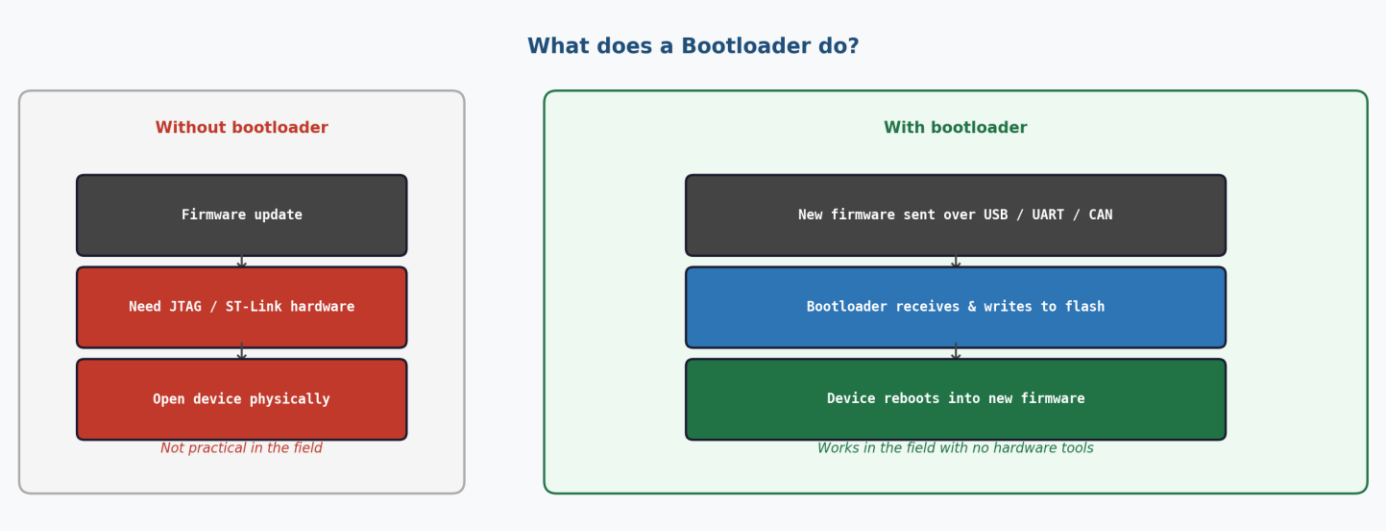

Figure 1 — Bootloader vs. no bootloader: field firmware update capability

Without a bootloader, updating firmware requires **physical access to the device** and a dedicated programmer (JTAG, SWD, or ST-Link). With a bootloader, updates can be delivered over **any communication bus** (USB, UART, CAN, Ethernet, or even wireless) — without opening the device or connecting any hardware.

NOTE NOTE	STM32 MCUs also have a built-in ROM bootloader programmed by ST at the factory. This document covers a custom user-written bootloader that lives in user flash, which gives you full control over the protocol, security, and behaviour.

2. How a Bootloader Fits in Flash Memory

All STM32 MCUs have internal flash memory starting at **`0x08000000`**. On every power-on or reset, the CPU automatically jumps to this address and begins executing. A bootloader takes advantage of this by placing itself at the very start of flash, so it is always the first code to run.

The flash is logically divided into two regions: the **bootloader region** (fixed at the start), and the **application region** (starting at a configurable address defined in your linker script). The boundary between them is entirely up to you — you choose it based on how large your bootloader is.

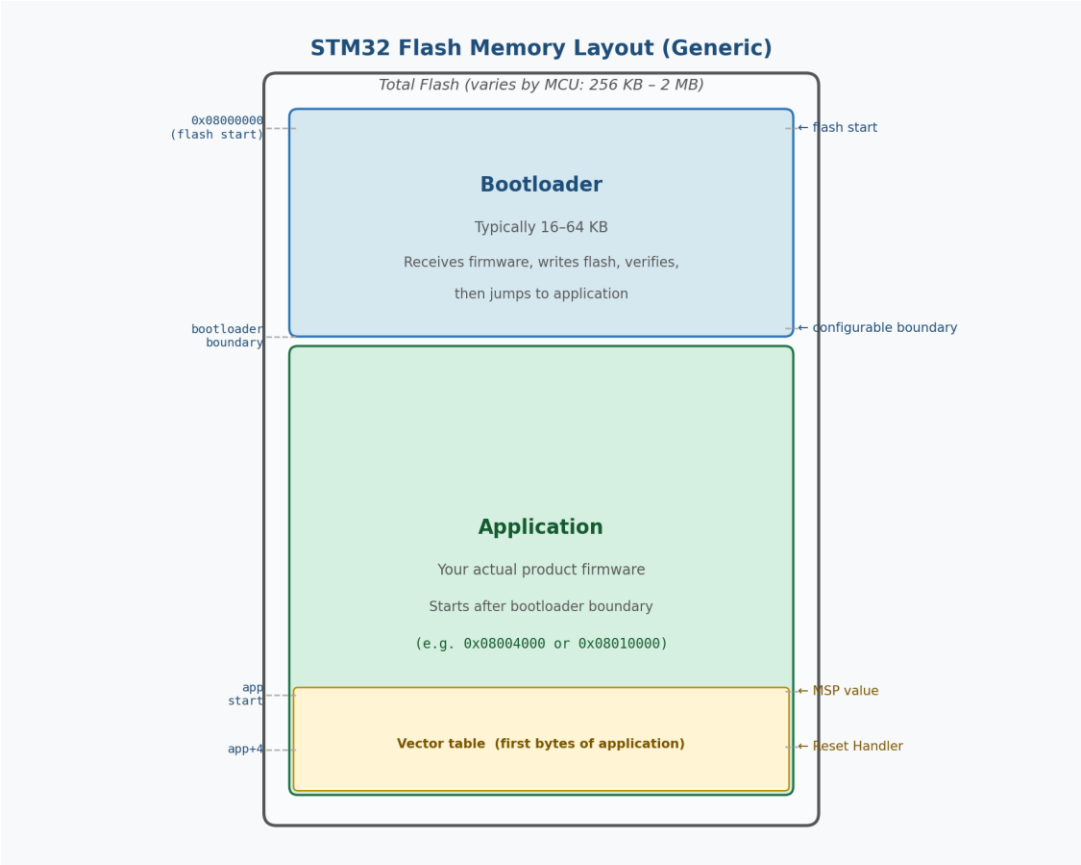

Figure 2 — Generic STM32 flash memory layout with bootloader and application regions

Item	Typical Value / Notes
Flash start (all STM32)	0x08000000 — fixed by silicon design
Bootloader size	16 KB – 64 KB depending on features
Application start	Configurable — e.g. 0x08004000, 0x08008000, 0x08010000
Application size	Remainder of flash minus bootloader
Vector table location	First bytes of the application region

3. Bootloader Execution Flow

Every time the MCU powers on or resets, it starts executing from the bootloader. The bootloader runs through a fixed sequence before either updating firmware or handing control to the existing application.

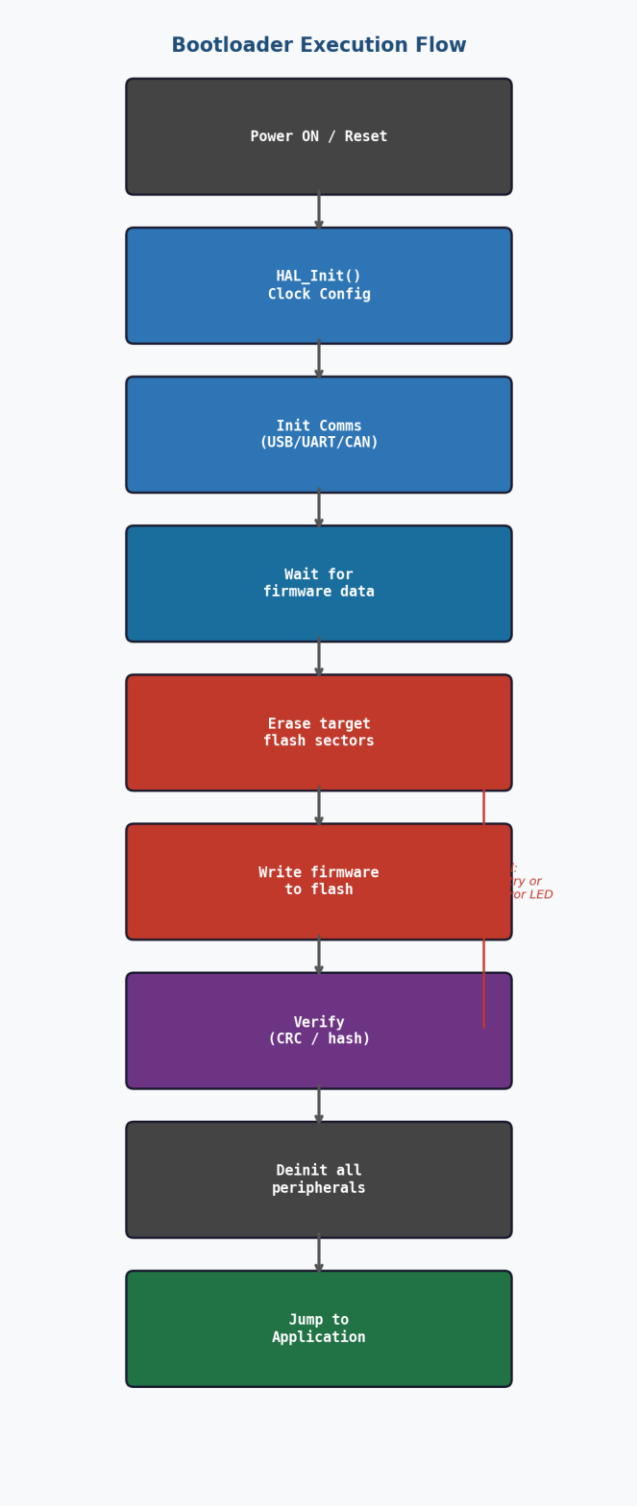

Figure 3 — Generic bootloader step-by-step execution flow

Step-by-Step Explanation

Step 1 — HAL Init and Clock Configuration

The bootloader calls **`HAL_Init()`** and **`SystemClock_Config()`** to bring the MCU into a known state. This configures the CPU clock speed, flash wait states, and prepares the hardware abstraction layer (HAL). Without this, peripheral drivers may not function correctly.

Step 2 — Initialise the Communication Interface

The bootloader initialises whichever communication peripheral it uses to receive firmware (USB, UART, CAN, SPI, etc.). It then waits — typically with a timeout — for an incoming firmware transfer. If no update arrives within the timeout window, it skips ahead and boots the existing application.

Step 3 — Erase Target Flash Sectors

Before writing new firmware, the target flash sectors must be **erased** to a known state (all bytes = 0xFF). Flash memory cannot be written without first erasing. The bootloader erases only the application region — never the bootloader region itself.

```
HAL_FLASH_Unlock();
FLASH_EraseInitTypeDef eraseInit = {
  .TypeErase = FLASH_TYPEERASE_SECTORS,
  .Sector    = APP_FIRST_SECTOR,   // sector where app starts
  .NbSectors = APP_SECTOR_COUNT,
  .VoltageRange = FLASH_VOLTAGE_RANGE_3
};
HAL_FLASHEx_Erase(&eraseInit, &sectorError);
```

Step 4 — Write Firmware to Flash

The received firmware bytes are written to flash in chunks. Different STM32 families have different write granularity requirements: F0/F1/L4 write in 4-byte words, F4/F7 in 1–4 bytes, H7 in **32-byte** flash words. Always align writes to the required boundary.

Step 5 — Verify the Written Firmware

After writing, the bootloader re-reads the flash and computes a **CRC-32** checksum (or compares against a hash) to confirm the data was written correctly. If verification fails, the bootloader can retry, erase the region, signal an error LED, or refuse to jump to the new firmware.

Step 6 — Deinit Peripherals and Jump

All peripherals initialised by the bootloader must be **de-initialised** before jumping to the application. If a peripheral (USB, SysTick, UART) is left running, its interrupt may fire in the application's context but the CPU will look up the handler from the wrong address — causing an immediate HardFault. After cleanup, the bootloader performs the jump sequence (detailed in Section 5).

4. Communication Interfaces

The bootloader can use any communication peripheral available on the STM32. The choice depends on your hardware, environment, and update frequency. The table below summarises the most common options:

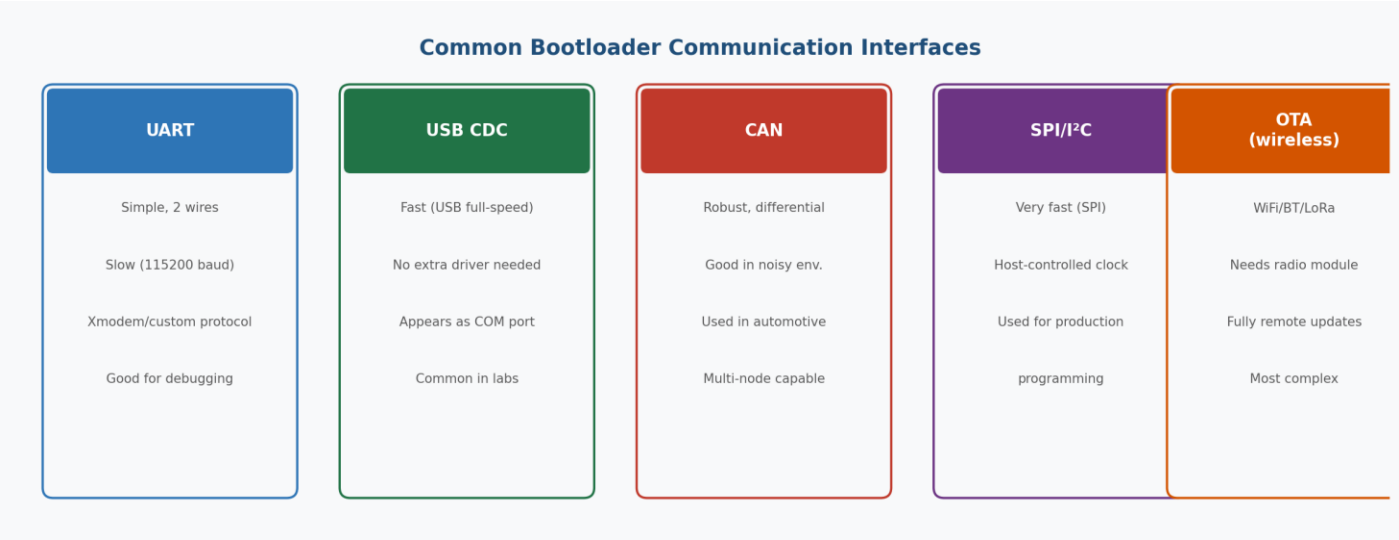

Figure 4 — Common bootloader communication interfaces comparison

Interface	Speed	Complexity	Best used when
UART	Slow (~115 Kbps)	Very low	Debugging, simple setups, lab use
USB CDC	Fast (12 Mbps)	Medium	PC-connected devices, lab and production
CAN	Medium (1 Mbps)	Medium	Automotive, industrial, noisy environments
SPI	Very fast	Low–Medium	Production programming, daisy-chained devices
Ethernet	Fast, long-range	High	Remote servers, industrial racks
OTA / RF	Wireless	High	IoT, fully remote field updates

TIP
TIP

UART is the simplest interface to start with when learning bootloader development. Once the write/verify/jump logic is working, switching to USB or CAN is mainly a change in the receive layer — the flash logic stays the same.

5. The Vector Table — How the Bootloader Finds the Application

When the bootloader wants to run the application, it needs to know two things: **where to put the stack** and **which function to call first**. Both answers are stored in the **vector table** — a small array of 4-byte pointers that the ARM toolchain automatically places at the very beginning of every application binary.

5.1 Structure of the Vector Table

The Cortex-M vector table is a fixed-format array starting at the application base address. The first two entries are critical for the jump:

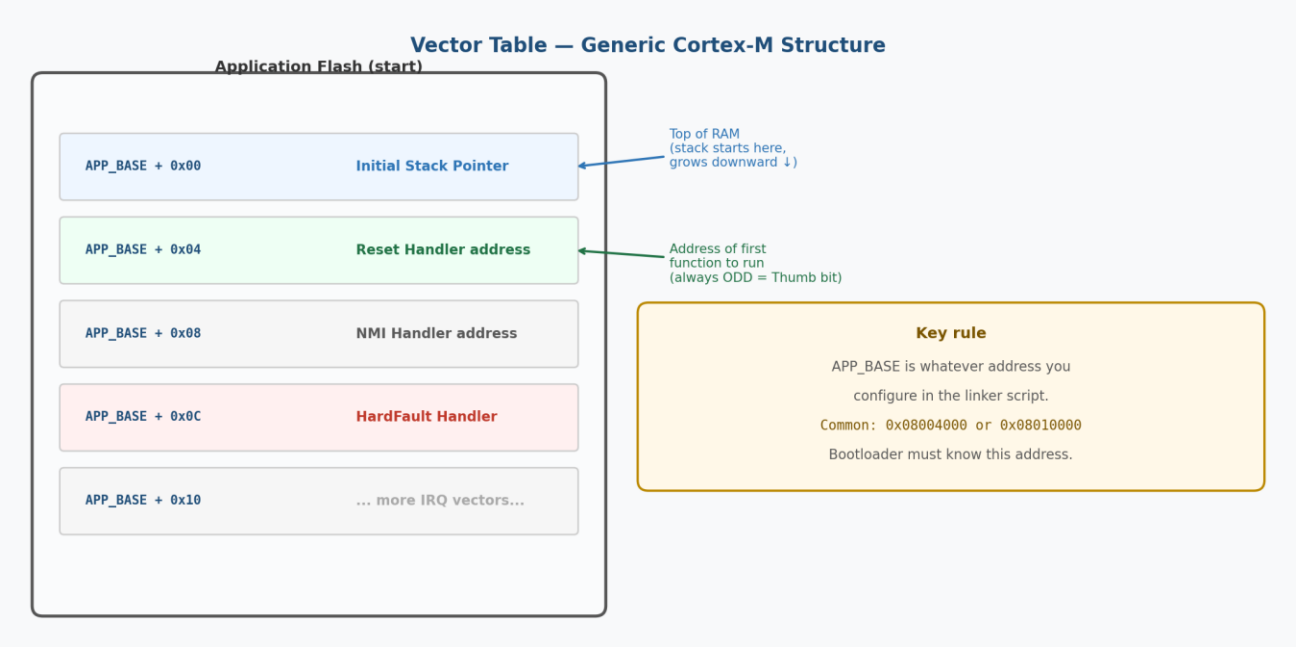

Figure 5 — Generic Cortex-M vector table structure

Offset	Field	Meaning
APP_BASE + 0x00	Initial Stack Pointer	Top of RAM — where the stack starts. Computed by the linker.
APP_BASE + 0x04	Reset Handler address	Address of the first function to run. Always ODD (Thumb bit set).
APP_BASE + 0x08	NMI Handler	Called on a Non-Maskable Interrupt.
APP_BASE + 0x0C	HardFault Handler	Called on unrecoverable CPU fault.
APP_BASE + 0x10 …	All other IRQ vectors	One entry per enabled interrupt in the application.

5.2 Who Writes These Values?

You never write these values manually. They are computed and placed by the **ARM toolchain** automatically during the build:

1. The **linker script (.ld file)** defines where flash and RAM are, and computes _estack (top of RAM).
2. The **startup assembly file (.s)** (provided by ST/CubeMX) lists every vector entry by name: .word _estack, .word Reset_Handler, etc.
3. The **linker** resolves every symbol name to its final address and writes the values — in little-endian byte order — into the binary.
4. The **binary is flashed to the MCU**, so the very first bytes at APP_BASE are these computed values.

5.3 How the Bootloader Reads Them

The bootloader reads the MSP and Reset Handler values with two simple C pointer dereferences:

```
#define APP_BASE_ADDRESS   0x08010000UL   // your chosen app start address

// Read Word [0] — Initial Stack Pointer
uint32_t app_msp = *(volatile uint32_t *) APP_BASE_ADDRESS;

// Read Word [1] — Reset Handler function pointer
void (*app_reset)(void) =
    (void (*)(void))(*(volatile uint32_t *)(APP_BASE_ADDRESS + 4));

// Sanity checks before jumping
if ((app_msp   & 0xFF000000) != 0x20000000) { error(); }  // must be RAM
if ((app_reset & 0xFF000000) != 0x08000000) { error(); }  // must be flash
if (app_msp == 0xFFFFFFFF)                  { error(); }  // flash not
programmed
```

KEY
KEY

APP_BASE_ADDRESS must match the ORIGIN address set in the application linker script. If they differ, the bootloader reads wrong values and the jump will fail or crash immediately.

5.4 Why is the Reset Handler Address Odd?

You will notice the Reset Handler address stored in the vector table is always an **odd number** (e.g. 0x08010101 instead of 0x08010100). This is intentional — it is the **Thumb bit**. The ARM Cortex-M architecture uses bit[0] of branch targets to indicate the instruction set: bit[0] = 1 means Thumb-2 (16/32-bit mixed), bit[0] = 0 means ARM32. All STM32 code is Thumb-2. The linker automatically sets bit[0] when writing the vector table. When the CPU branches to this address (via BX), it strips bit[0] to get the real even address and switches to Thumb mode.

6. Jump-to-Application Sequence

The jump from bootloader to application must be performed in a precise order. Any peripheral left running by the bootloader can cause an immediate HardFault in the application because its interrupt will fire but the CPU will look up the handler from the wrong vector table address.

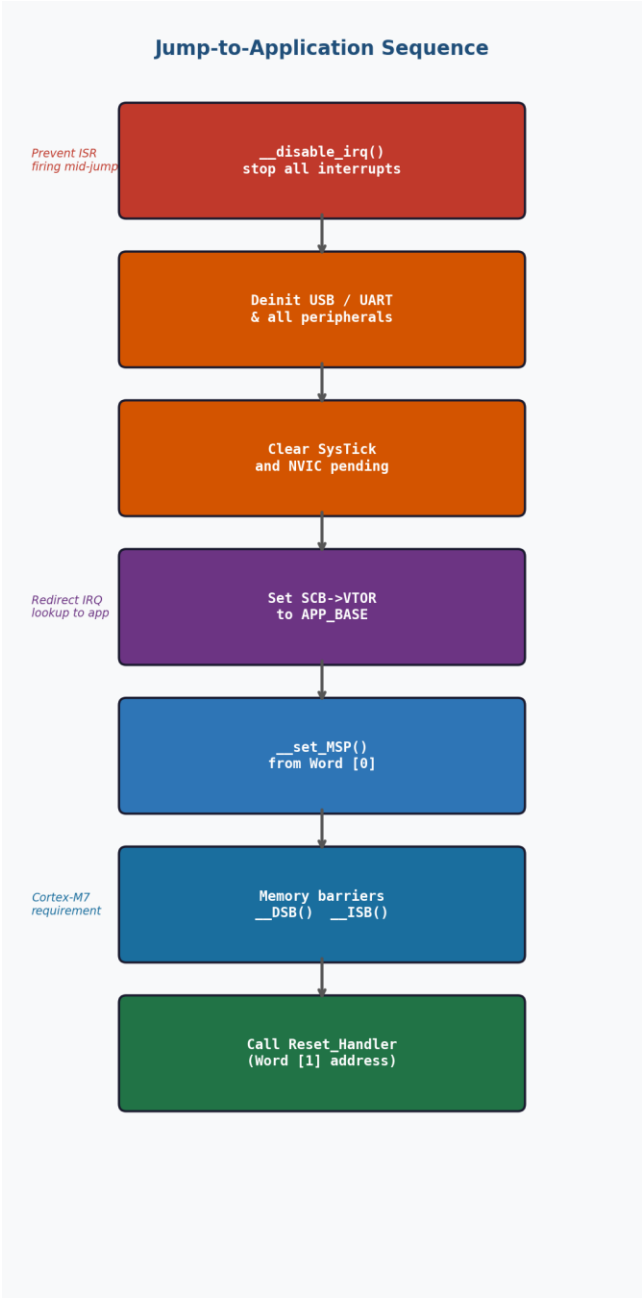

Figure 6 — Jump-to-application step sequence

6.1 Complete Jump Code

```
void jump_to_application(uint32_t app_base)
{
    uint32_t app_msp    = *(volatile uint32_t *) app_base;
    uint32_t app_reset  = *(volatile uint32_t *)(app_base + 4);

    // Sanity check
    if ((app_msp & 0xFF000000) != 0x20000000) return;

    // 1. Stop all interrupts
    __disable_irq();

    // 2. De-initialise all peripherals used by the bootloader
    HAL_DeInit();              // resets HAL state
    // also call e.g. HAL_PCD_DeInit(), HAL_UART_DeInit() as needed

    // 3. Stop the system tick and clear NVIC
    SysTick->CTRL = 0;
    for (int i = 0; i < 8; i++) {
        NVIC->ICER[i] = 0xFFFFFFFF;   // disable all IRQs
        NVIC->ICPR[i] = 0xFFFFFFFF;   // clear pending
    }

    // 4. Redirect vector table to application
    SCB->VTOR = app_base;

    // 5. Load application stack pointer
    __set_MSP(app_msp);

    // 6. Memory barriers (important on Cortex-M7)
    __DSB();
    __ISB();

    // 7. Jump!
    void (*reset_fn)(void) = (void (*)(void)) app_reset;
    reset_fn();
}
```

6.2 Why Each Step Matters

Step	Why it is necessary
__disable_irq()	Stops any interrupt from firing during the transition — prevents CPU jumping to bootloader ISR mid-switch
HAL_DeInit() + peripheral deinit	Releases clock gates, resets peripheral registers — leaves hardware in a clean state for the application
SysTick->CTRL = 0	Stops the 1 ms system tick. If left running, it fires every millisecond but the application has not set up its handler yet

Step	Why it is necessary
NVIC ICER + ICPR	Disables and clears all pending IRQs. Without this, a pending USB or DMA interrupt fires the moment __enable_irq() is called
SCB->VTOR = app_base	Tells the CPU where the new vector table is. Without this, all interrupts still look up handlers from the bootloader address
__set_MSP(app_msp)	Loads the application stack. Without this the app continues using the bootloader stack and will corrupt it immediately
__DSB() + __ISB()	Memory and instruction barriers — flushes the write to VTOR before the branch (required on Cortex-M7, good practice everywhere)
reset_fn()	Branches to the application Reset_Handler. Application starts — bootloader is done

7. Flash Write Rules by STM32 Family

Different STM32 families have different flash write granularity requirements. Using the wrong write size causes a **`FLASH_ERROR_PROGRAM`** or simply writes 0xFF everywhere. Always match the **`TypeProgram`** value to your MCU family:

STM32 Family	TypeProgram	Write size	Notes
F0, F1, F3	FLASH_TYPEPROGRAM_HALFWORD	2 bytes	Low-density flash
F2, F4, F7	FLASH_TYPEPROGRAM_WORD	4 bytes	Sector-based erase
H7	FLASH_TYPEPROGRAM_FLASHWORD	32 bytes	Must align to 32-byte boundary
L0, L1	FLASH_TYPEPROGRAM_HALFPAGE	16 bytes	Low-power flash, half-page write
L4, G0, G4, U5	FLASH_TYPEPROGRAM_DOUBLEWORD	8 bytes	Dual-bank option on some
WB, WL	FLASH_TYPEPROGRAM_DOUBLEWORD	8 bytes	Wireless MCUs, same flash IP

WARN
WARN

Always call HAL_FLASH_Unlock() before writing and HAL_FLASH_Lock() after. Flash protection (WRP — Write Read Protection) can be configured per-sector. The bootloader sector itself should be write-protected so a buggy application cannot erase it.

8. Common Errors and How to Debug Them

Symptom	Root cause	Fix
HardFault immediately after jump	VTOR not set, or SysTick/USB left running	Set SCB->VTOR, call HAL_DeInit(), zero SysTick->CTRL, clear NVIC before jump
PC jumps to 0xFFFFFFFF	Flash not programmed — reading erased flash (all 0xFF)	Check sanity: app_msp should be in 0x20xxxxx range, not 0xFFFFFFFF
Application starts but crashes in first ISR	NVIC not cleared — pending interrupt fires with old vector table	Clear NVIC->ICPR registers before the jump
Flash write fails silently	Wrong TypeProgram for the MCU family, or write not 32-byte aligned (H7)	Use objdump to verify flash content; check alignment
Bootloader corrupts itself	Erasing wrong sectors — app and bootloader sectors overlap in config	Double-check APP_FIRST_SECTOR matches your linker script ORIGIN
USB not working in application after bootloader	USB not fully de-initialised before jump	Call HAL_PCD_DeInit() and HAL_PWREx_DisableUSBVoltageDetector() before jump

9. Linker Script Configuration

Both the bootloader and the application need linker scripts that agree on the memory boundary. The most common mistake is a mismatch between what the bootloader thinks APP_BASE_ADDRESS is and where the application is actually linked.

9.1 Bootloader Linker Script

The bootloader linker script places the bootloader at the start of flash. The size should be just large enough to fit the bootloader binary with some margin:

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 64K   /* bootloader region */
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K  /* adjust for your MCU
*/
}
```

9.2 Application Linker Script

The application must start at exactly the address the bootloader knows about. This is usually set in the **MEMORY** block of the application linker script:

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08010000, LENGTH = 1984K  /* after 64 KB
bootloader */
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
```

KEY
KEY

The ORIGIN of FLASH in the application linker script must exactly match APP_BASE_ADDRESS defined in the bootloader. Also, if the application uses the VTOR (SCB->VTOR), it must be set in SystemInit() or at startup — not left pointing to 0x08000000.

10. Implementation Checklist

Use this checklist when implementing a bootloader from scratch:

#	Task
1	Set FLASH ORIGIN in bootloader linker script to 0x08000000
2	Set FLASH ORIGIN in application linker script to your chosen APP_BASE
3	Define APP_BASE_ADDRESS constant in bootloader code to match application ORIGIN
4	Unlock flash (HAL_FLASH_Unlock) before any erase or write operation
5	Erase only the application sectors — never the bootloader sector
6	Use the correct TypeProgram for your STM32 family (see Section 7)
7	Verify written firmware with CRC-32 or byte comparison before jumping
8	Read Word[0] (MSP) and Word[1] (Reset Handler) from APP_BASE
9	Validate MSP is in RAM range (0x20000000) and not 0xFFFFFFFF
10	Call __disable_irq() before starting the jump sequence
11	De-initialise all peripherals: USB, UART, CAN, SPI, timers
12	Clear SysTick->CTRL = 0 and clear all NVIC ICER/ICPR registers
13	Set SCB->VTOR = APP_BASE before setting MSP
14	Call __set_MSP(app_msp) to load the application stack pointer
15	Call __DSB() and __ISB() for memory and instruction barriers
16	Branch to the application Reset Handler function pointer
17	(Optional) Write-protect the bootloader flash sector with WRP
18	(Optional) Add a hardware entry pin — hold low = stay in bootloader

11. Summary

A bootloader on an STM32 is a small, always-first piece of firmware that enables field firmware updates over any communication bus. It occupies the start of flash (`0x08000000`), while the application lives at a configurable address further into flash. The boundary between the two is set in the linker scripts of both projects and must match exactly.

The bootloader's job is straightforward: receive new firmware, erase the application flash region, write and verify the new binary, then jump cleanly to the application. The jump requires a precise teardown sequence — disable interrupts, deinit peripherals, clear SysTick and NVIC, set VTOR, load MSP, then branch — to guarantee the application starts in a clean hardware state.

The two values needed for the jump (MSP and Reset Handler address) are read directly from the application's **vector table** — the small array of function pointers that the ARM toolchain automatically places at the very beginning of every application binary. These are not written manually; they are computed by the linker from the linker script's memory definitions and the startup assembly file's function addresses.

Concept	One-line summary
Bootloader purpose	Receive, write, and verify new firmware — then launch the application
Memory layout	Bootloader at 0x08000000; app at configurable boundary; both in linker scripts
Vector table	Array of pointers at app start: Word[0] = MSP, Word[1] = Reset Handler
Thumb bit	Reset Handler address is always odd — linker adds +1; CPU strips it before branching
Jump sequence	Disable IRQ → deinit → clear NVIC → set VTOR → set MSP → barriers → call fn
Flash write	Erase first, then write with correct TypeProgram for your STM32 family
Verification	CRC-32 after write; sanity check MSP/reset values before jump
Linker scripts	APP_BASE in bootloader code must match FLASH ORIGIN in application .ld file